HDOC Day-54

Name	Gurveer Singh Cheema
SAP ID	590032925
Batch	15

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array **arr[]**, and an integer **k**. Pass the array to a function named **max_subarray_sum()** using its address. Use pointers to pass the array between the calling function and **max_subarray_sum()**.

The function must identify all contiguous subarrays of size **k**, calculate the sum of each subarray, and display the maximum subarray sum. If **k** is invalid, display **-1**.

C Program

```
#include <stdio.h>

void max_subarray_sum(int *arr, int size, int k)
{
    int i, j, sum, maxSum;

    if (k <= 0 || k >= size)
    {
        printf("-1\n");
        return;
    }

    maxSum = 0;
    for (j = 0; j < k; j++)
        maxSum += *(arr + j);

    for (i = 1; i <= size - k; i++)
    {
        sum = 0;

        for (j = i; j < i + k; j++)
            sum += *(arr + j);

        if (sum > maxSum)
            maxSum = sum;
    }

    printf("Maximum subarray sum: %d\n", maxSum);
}

int main()
{
    int size, i, k;

    printf("Enter size of array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter array elements: ");
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Enter k: ");
    scanf("%d", &k);

    max_subarray_sum(arr, size, k);

    return 0;
}
```

Explanation

The array address is passed to **max_subarray_sum()** using a pointer. The program first checks whether **k** is valid. It then considers every contiguous group of **k** elements, calculates its sum using pointer notation, and compares it with the current maximum. Finally, the largest sum is displayed.

Sample Outputs

```
Example 1
Array: 2 1 5 1 3 2
k: 3
Maximum subarray sum: 9

Example 2
Array: 2 3 4 1 5
k: 2
Maximum subarray sum: 7

Example 3
Array: 1 2 3
k: 4
-1
```

Result

Thus, the program successfully finds the maximum sum among all contiguous subarrays of size k using pointers and displays -1 when k is invalid.